

Protocole de communication

Contrôleur d'Arrosage - Version 1.2

1. Objectif

Ce protocole définit les échanges JSON via WebSocket entre l'application Qt/QML et l'ESP32. Il est compact, extensible et optimisé pour un traitement simple avec ArduinoJson côté ESP32.

2. Structure générale

```
{"v":1,"t":"<type>","c":"<commande>","... }
```

Clé	Description	Type
v	Version du protocole	int
t	Type de trame	char
c	Commande	char
i	Identifiant vanne	int
m	Mode	char
e	Etat vanne	char
h	Date/heure ISO8601	string
d	Durée (minutes)	int
f	Fréquence (heures)	int
x	Code erreur	int

3. Types de trames

q : Query (demande)
r : Response (réponse)
c : Commande
a : Ack (accusé de réception)
e : Erreur
n : Notification spontanée

4. Commandes

T = get_Time, t = set_Time
E = get_Etat, M = get_Mode, m = set_Mode
P = get_Prog, p = set_Prog
O = Ouvrir, F = Fermer
G = Ping

5. Modes et états

Modes : M = Manuel, P = Programmé, A = Automatique
États : "F" = Fermée, "O" = Ouverte

6. Cas d'utilisation

Demande heure

Requête : {"v":1,"t":"q","c":"T"}

Réponse : {"v":1,"t":"r","c":"T","h":"2026-07-08T18:30:00"}

Mise à jour heure

Requête : {"v":1,"t":"c","c":"t","h":"2026-07-08T18:30:00"}

Réponse : {"v":1,"t":"a","c":"t"}

Demande état

Requête : {"v":1,"t":"q","c":"E","i":1}

Réponse : {"v":1,"t":"r","c":"E","i":1,"e":"F"}

Demande mode

Requête : {"v":1,"t":"q","c":"M","i":1}

Réponse : {"v":1,"t":"r","c":"M","i":1,"m":"P"}

Mise à jour mode

Requête : {"v":1,"t":"c","c":"m","i":1,"m":"A"}

Réponse : {"v":1,"t":"a","c":"m","i":1}

Demande programmation

Requête : {"v":1,"t":"q","c":"P","i":1}

Réponse : {"v":1,"t":"r","c":"P","i":1,"h":"2026-07-08T18:30:00","d":15,"f":24}

Mise à jour programmation

Requête : {"v":1,"t":"c","c":"p","i":1,"h":"2026-07-08T18:30:00","d":15,"f":24}

Réponse : {"v":1,"t":"a","c":"p","i":1}

Ouverture vanne

Requête : {"v":1,"t":"c","c":"O","i":1}

Réponse : {"v":1,"t":"a","c":"O","i":1}

Fermeture vanne

Requête : {"v":1,"t":"c","c":"F","i":1}

Réponse : {"v":1,"t":"a","c":"F","i":1}

Ping

Requête : {"v":1,"t":"q","c":"G"}

Réponse : {"v":1,"t":"r","c":"G"}

[v1.1] Demande état complet du système

Requête : {"v":1,"t":"q","c":"S"}

Réponse : {"v":1,"t":"r","c":"S","h":"2026-07-08T18:30:00",
"s":[{"i":1,"e":"F","m":"P","h":"2026-07-08T18:30:00","d":15,"f":24},
{"i":2,"e":"O","m":"M","h":"2026-07-08T06:00:00","d":30,"f":12}]}

La clé h de premier niveau contient l'heure système de l'ESP32 (contrôle de dérive de la RTC). La clé s contient un tableau dont chaque élément décrit une vanne en réutilisant les clés i, e, m, h, d, f définies au §2. Cette commande est utilisée par l'application à chaque (re)connexion pour synchroniser l'image logicielle des vannes avec l'état réel, en une seule trame quel que soit le nombre de vannes.

7. Notifications spontanées ESP32 → Application mobile

Les notifications sont envoyées lorsqu'un évènement se produit sur l'ESP32 (programmation exécutée, changement d'état, changement de mode, etc.). Elles sont asynchrones et ne nécessitent aucun acquittement. Le smartphone peut être connecté ou non, et l'ESP32 reste maître de son fonctionnement autonome.

Vanne ouverte automatiquement (programmation ou auto) :

```
{"v":1,"t":"n","c":"O","i":1,"e":"F"}
```

Vanne fermée automatiquement (programmation ou auto) :

```
{"v":1,"t":"n","c":"F","i":1,"e":"O"}
```

Mode modifié localement sur l'ESP32 :

```
{"v":1,"t":"n","c":"M","i":1,"m":"A"}
```

Programmation modifiée localement sur l'ESP32 :

```
{"v":1,"t":"n","c":"P","i":1,"h":"2026-07-08T18:30:00","d":15,"f":24}
```

[v1.2] Notification de mise en veille :

```
{"v":1,"t":"n","c":"V"}
```

8. Gestion des erreurs

Format : {"v":1,"t":"e","c":"<commande>","x":"<code>"}

1 : Vanne inexistante

2 : Mode invalide

3 : Programmation invalide

4 : RTC indisponible

5 : Base SQLite indisponible